UABC

Título de la práctica:

Práctica 3. Ciclo for



Nombre del alumno(s):

Avendaño García Axel Emilio

Matrícula del alumno(s):

2113250

Nombre del docente:

Joscelyn Pauleth Covarrubias Murrieta

Ciudad y fecha de entrega:

Mexicali, Baja California, 17 de septiembre de 2026

Introducción

Los ciclos permiten realizar operaciones repetitivas dentro de los programas. En C hay tres tipos de ciclos: el ciclo for, el do-while y el while. En esta práctica los ejercicios que se realicen se resolverán utilizando el ciclo for.

Objetivo

Utilizar ciclos apropiadamente para realizar operaciones repetitivas dentro de programas mostrando disposición por la investigación.

Fundamento

El ciclo for se utiliza cuando se conoce de antemano cuántas veces se deberán repetir las instrucciones, ya que en su cabecera se agrupan la inicialización de la variable de control, la condición de repetición y el incremento o decremento, todo en una sola línea: for (inicialización; condición; incremento). Mientras la condición se evalúe como verdadera, el bloque de instrucciones se seguirá ejecutando; en cuanto la condición sea falsa, el ciclo termina.

A diferencia del while y el do-while, que son ciclos de repetición inexacta (pueden iterar un número de veces que depende de una condición externa), el for es un ciclo de repetición exacta, ideal para recorrer rangos numéricos conocidos, como generar los términos de una serie o dibujar figuras con un número definido de filas y columnas. También se pueden anidar varios ciclos for para controlar renglones y columnas al mismo tiempo, como se hizo en el ejercicio de las figuras.

Para el ejercicio de la carrera de dados se utilizó la función rand(), perteneciente a la librería stdlib.h, la cual genera un número entero pseudoaleatorio. Para acotar el resultado a un rango determinado se utiliza el operador módulo (%); por ejemplo, la expresión rand() % (max - min + 1) + min genera un número entero entre min y max. También se utilizó la función srand(), incluida en stdlib.h, para establecer una semilla de generación distinta en cada ejecución del programa, usando como semilla la hora actual obtenida con time(NULL) de la librería time.h.

Las librerías utilizadas en los programas de esta práctica fueron stdio.h (funciones de entrada y salida como printf() y scanf()), stdlib.h (función rand() y srand()) y time.h (función time(), usada junto con srand()).

Procedimiento (Desarrollo de la práctica)

Ejercicio Presencial (20 puntos)

1. Elabora un programa en C que funcione como la función pow del archivo de cabecera math.h. Su programa primero solicitará la base y posteriormente el exponente para proceder a mostrar el resultado. Considerar exponentes negativos.

Código:

```c
#include <stdio.h>

int main() {

 float pow, n, resultado = 1;

 printf("Ingresa un numero: ");
 scanf("%f", &n);
 printf("Ahora ingresa la potencia que quieres calcular: ");
 scanf("%f", &pow);
 if (pow >= 0) {
   for (int i = 0; i < pow; i++) {
     resultado = resultado * n;
   }
 } else {
   for (int i = 0; i < -pow; i++) {
     resultado *= n;
   }
   resultado = 1 / resultado;
 }
 printf("El resultado es de: %f \n", resultado);
 return 0;
}
```

Ejecución del programa:

```
Ingresa un numero: 5
Ahora ingresa la potencia que quieres calcular: 2
El resultado es de: 25.000000
```

Descripción del programa: El programa solicita al usuario una base (n) y un exponente (pow). Si el exponente es mayor o igual a cero, se utiliza un ciclo for que multiplica resultado por n tantas veces como indique el exponente. Si el exponente es negativo, el ciclo for multiplica resultado por n usando el valor absoluto del exponente (-pow) y, al terminar, se calcula el recíproco (1 / resultado) para obtener la potencia negativa. De esta forma el programa reproduce el comportamiento de la función pow() de math.h, incluyendo el caso de exponentes negativos.

Ejercicios para Llevar (80 puntos)

2. Escribe un programa que muestre por separado cada una de las 6 figuras mostradas. Éstas deberán generarse usando ciclos for. Esto significa que no es aceptable generar las figuras utilizando uno o varios printf() que tengan los caracteres de manera estática.

Código: En Anexos

Ejecución del programa:

```
I)
**********
**********
**********
**********

II)
*
**
***
****
*****

III)
    *
   **
  ***
 ****
*****
```

```
IV)
    *
   ***
  *****
 *******
*********

V)
    1
   222
  33333
 4444444
555555555

VI)
    1
   212
  32123
 4321234
543212345
```

Descripción del programa:

El programa dibuja las seis figuras utilizando ciclos for anidados, sin escribir los caracteres de forma estática. Para la figura I se recorre un ciclo de renglones y, dentro de él, un ciclo de columnas que imprime asteriscos. Para la figura II el número de asteriscos por renglón depende del índice del renglón (a mayor renglón, más asteriscos). Las figuras III y IV combinan un ciclo que imprime espacios en blanco para generar la sangría junto con un segundo ciclo que imprime los asteriscos, ajustando la cantidad de cada uno según el renglón para formar el triángulo y el árbol. Las figuras V y VI utilizan la misma lógica de sangría, pero en lugar de asteriscos imprimen números: en la figura V se repite el número del renglón, mientras que en la figura VI se imprime primero una secuencia descendente y luego una ascendente para formar el patrón numérico.

3. Escribe un programa en C que muestre los n-términos de la serie de Fibonacci. Deberá validar que la cantidad de términos a mostrar sea un número entre 1 y 100.

Código:

```c
#include <stdio.h>

int main() {

 long a = 0, b = 1, c = 0, n = 0;

 printf("Ingresa n numeros de la secuencia Fibonacci: ");
 scanf("%ld", &n);

 if ((n <= 100) && (n >= 1)) {
   for (int i = 0; i < n; i++) {
     c = a + b;
     a = b;
     b = c;
     printf("%ld ", a);
   }
 } else {
```

```
   printf("Ingresa un numero valido (1-100)");
 }
 return 0;
}
```

Ejecución:

```
Ingresa n numeros de la secuencia Fibonacci: 10
1 1 2 3 5 8 13 21 34 55
```

Descripción:

El programa solicita al usuario cuántos términos de la serie de Fibonacci desea generar. Primero se valida que el valor ingresado esté dentro del rango permitido (entre 1 y 100); si no lo está, se muestra un mensaje de error y el programa termina sin generar la serie. Si el valor es válido, un ciclo for recorre n veces, calculando en cada iteración el siguiente término como la suma de los dos anteriores (c = a + b) y actualizando las variables a y b para la siguiente iteración, imprimiendo cada término conforme se genera.

4. Escribe un programa en C que simule un juego de dados entre dos jugadores durante un determinado número de rondas dado por el usuario.

○ Al inicio de cada ronda, el programa deberá generar dos números enteros aleatorios (utilizar ciclo for en caso de que se quieran utilizar más dados) entre 1 y 6 simulando la tirada de dos dados para cada jugador. (Investigar la función rand() para números aleatorios).
○ El jugador con el número más alto gana la ronda y se suma un punto a su marcador personal, en caso de empate, ningún jugador sumará puntos.
○ Al finalizar el número de rondas, se deberá imprimir la puntuación acumulada de cada jugador y lanzar un mensaje de felicitación para el ganador o en dado caso, un mensaje de empate.Suma 10 puntos a cada jugador.

Código: En Anexos

Ejecución:

```
===== CARRERA DE DADOS =====
Ingresa un numero de rondas: 3

--- Ronda 1 de 3 ---
Jugador 1 lanza: [3][1]-> Total J1: 4
Jugador 2 lanza: [3][2]-> Total J2: 5
-> Ganador de la ronda: Jugador 2 (+1 pt)

--- Ronda 2 de 3 ---
Jugador 1 lanza: [6][5]-> Total J1: 11
Jugador 2 lanza: [6][3]-> Total J2: 9
-> Ganador de la ronda: Jugador 1 (+1 pt)

--- Ronda 3 de 3 ---
Jugador 1 lanza: [5][4]-> Total J1: 9
Jugador 2 lanza: [2][1]-> Total J2: 3
-> Ganador de la ronda: Jugador 1 (+1 pt)

================ RESUMEN FINAL ================
```

Descripción:

El programa solicita al usuario el número de rondas que durará la partida. Un ciclo for externo recorre cada ronda y, dentro de éste, dos ciclos for internos generan dos números aleatorios entre 1 y 6 para cada jugador (simulando la tirada de dos dados), acumulando el total de cada uno y mostrando entre corchetes cada tirada individual conforme se genera. Se compara el total de ambos jugadores: si el de Jugador 1 es mayor, éste gana la ronda y suma un punto; si el de Jugador 2 es mayor, gana él; y si son iguales, la ronda es un empate y ningún jugador suma puntos. Al terminar todas las rondas se imprime el marcador final y, dependiendo de quién tenga más puntos, se muestra un mensaje de felicitación para el ganador o un mensaje de empate global.

Conclusiones

En esta práctica se aprendió a utilizar el ciclo for para realizar operaciones repetitivas dentro de programas en lenguaje C. Se comprendió que este ciclo es ideal cuando se conoce de antemano el número de repeticiones, ya que agrupa en una sola línea la inicialización, la condición y el incremento de la variable de control, a diferencia de los ciclos while y do-while, que dependen de una condición externa y son de repetición inexacta.

Los ejercicios permitieron aplicar estos conocimientos en distintas situaciones: la implementación de una función que emula a pow() de math.h considerando exponentes negativos, la generación de figuras y patrones numéricos utilizando ciclos for anidados, la construcción de la serie de Fibonacci validando el rango de entrada, y la simulación de un juego de dados entre dos jugadores utilizando la función rand() para generar números aleatorios. También se reforzó el uso de estructuras if, else if, así como operadores aritméticos y lógicos para controlar el flujo de los programas.

Referencias

- Covarrubias Murrieta, J. P. (2026). *Material de clase de la asignatura de programación*. Universidad Autónoma de Baja California.
- GeeksforGeeks. (s. f.). *Generating random number in a range in C*. https://www.geeksforgeeks.org/c/generating-random-number-range-c/
- Ogo. (s. f.). *Generar números aleatorios en C* [Video]. YouTube. https://www.youtube.com/watch?v=Ogo_DpLhQl0

Anexos

Ejercicio 2:

```
#include <stdio.h>

int main() {
 // I
 printf("I)\n");
 for (int rows = 0; rows < 4; rows++) {
   for (int columns = 0; columns < 10; columns++) {
     printf("*");
   }
   printf("*\n");
 }

 // II
 printf("\n");
 printf("II)\n");

 for (int i = 0; i < 5; i++) {
   for (int j = 0; j < i; j++) {
     printf("*");
   }
   printf("*\n");
 }

 printf("\n");
 printf("III)\n");

 // III
 for (int i = 5; i > 0; i--) {
   for (int j = 1; j < i; j++) {
     printf(" ");
   }
   for (int k = 0; k < 6 - i; k++) {
     printf("*");
   }
   printf("\n");
 }

 printf("\n");
 printf("IV)\n");
```

```
 // IV
 for (int i = 5; i > 0; i--) {
   for (int j = 1; j < i; j++) {
     printf(" ");
   }
   for (int k = 0; k < (11 - 2 * i); k++) {
     printf("*");
   }
   printf("\n");
 }

 printf("\n");
 // IV
 printf("V)\n");
 for (int i = 1; i <= 5; i++) {
   for (int j = 5; j > i; j--) {
     printf(" ");
   }
   for (int k = 0; k < 2 * i - 1; k++) {
     printf("%i", i);
   }
   printf("\n");
 }
 printf("\n");
 printf("VI)\n");

 for (int i = 1; i <= 5; i++) {
   for (int j = 5; j > i; j--) {
     printf(" ");
   }
   for (int k = i; k >= 1; k--) {
     printf("%i", k);
   }
   for (int k = 2; k <= i; k++) {
     printf("%i", k);
   }
   printf("\n");
 }
 return 0;
}
```

Ejercicio 4:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {

 srand(time(NULL));

 int min = 1, max = 6;
 int dado_j1 = 0, dado_j2 = 0;
 int p1_score = 0, p2_score = 0, rondas = 0;

 printf("===== CARRERA DE DADOS =====\n");
 printf("Ingresa un numero de rondas: ");
 scanf("%d", &rondas);
 printf("\n");
 for (int i = 1; i <= rondas; i++) {
   printf("--- Ronda %d de %d ---\n", i, rondas);

   printf("Jugador 1 lanza: ");
   for (int j = 0; j < 2; j++) {
     int print_temp = rand() % (max - min + 1) + min;
     dado_j1 += print_temp;
     printf("[%d]", print_temp);
   }
   printf("-> Total J1: %d\n", dado_j1);
   printf("Jugador 2 lanza: ");
   for (int j = 0; j < 2; j++) {
     int print_temp = rand() % (max - min + 1) + min;
     dado_j2 += print_temp;
     printf("[%d]", print_temp);
   }
   printf("-> Total J2: %d\n", dado_j2);

   if (dado_j1 > dado_j2) {
     p1_score++;
     printf("-> Ganador de la ronda: Jugador 1 (+1 pt)\n");
     printf("\n");
   } else if (dado_j1 < dado_j2) {
     p2_score++;
     printf("-> Ganador de la ronda: Jugador 2 (+1 pt)\n");
     printf("\n");
```

```
    } else {
      printf("->Empate en esta ronda (Nadie suma puntos)\n");
      printf("\n");
    }

    dado_j1 = 0;
    dado_j2 = 0;
  }
  printf("================ RESUMEN FINAL ================\n"
         "Puntaje Jugador 1: %d\n"
         "Puntaje Jugador 2: %d\n",
         p1_score, p2_score);
  if (p1_score > p2_score) {
    printf("FELICDADES AL JUGADOR 1!!!\n");
  } else if (p1_score < p2_score) {
    printf("FELICIDADES AL JUGADOR 2!!!\n");
  } else {
    printf("¡LA PARTIDA TERMINÓ EN EMPATE GLOBAL!\n");
  }
  printf("==============================================");

  return 0;
}
```